# Week 2

# Part 1

```
fasetid2@cheetah:~$ time ls
HPQC  PHY1090

real    0m0.006s
user    0m0.000s
sys     0m0.005s
fasetid2@cheetah:~$
```

# Part 2

### Timing hello world programs

```
fasetid2@cheetah:~$ time python3 ./PHY1090/week2/hello_world.py
Hello, World!

real    0m0.033s
user    0m0.021s
sys     0m0.009s
fasetid2@cheetah:~$ time ./bin/hello_world
Hello, World!

real    0m0.006s
user    0m0.000s
sys     0m0.004s
```

### Timing repeat adder programs

```
fasetid2@cheetah:~$ time bin/repeat_adder 2 3
6

real    0m0.005s
user    0m0.004s
sys     0m0.000s
fasetid2@cheetah:~$ time python3 ./PHY1090/week2/repeat_adder.py 4 5
20


real    0m0.030s
user    0m0.025s
sys     0m0.005s
```

```
fasetid2@cheetah:~$ time bin/repeat_adder 100 780
78000

real    0m0.004s
user    0m0.000s
sys     0m0.004s
fasetid2@cheetah:~$ time python3 ./PHY1090/week2/repeat_adder.py 500 960
480000


real    0m0.029s
user    0m0.016s
sys     0m0.012s
```

| Input (in_arg, multiple) | Output | Real time |
|---|---|---|
| (100000, 1) | 100000 | 0.037 s |
| (100000, 10) | 1000000 | 0.026 s |
| (100000, 100) | 10000000 | 0.032 s |
| (100000, 1000) | 100000000 | 0.031 s |

| Input (in_arg, multiple) | Output | Real time |
|---|---|---|
| (100000, 10) | 1000000 | 0.005 s |
| (100000, 100) | 10000000 | 0.004 s |
| (100000, 1000) | 100000000 | 0.003 s |
| (100000, 10000) | 1000000000 | 0.005 s |

The timing results show that varying the second argument had little effect on runtime, because the program's loop depends on the first argument. Since the first argument was fixed at 100000 in all cases, the number of loop iterations stayed the same. The output increased as the second argument increased, but the runtime remained nearly constant. The C implementation was consistently faster than the Python version, showing the performance advantage of compiled code over interpreted code.

The results also show that the **C version is faster than the Python version**. The Python runtimes were around `0.026-0.037 s`, while the C runtimes were around `0.003-0.005 s`. This suggests that compiled C code executes the repeated-addition loop more efficiently than interpreted Python.

# Part 3

| Input Size | C Runtime (s) | Python Runtime (s) |
|---|---|---|
| 10 | 0.000159 | 0.0000138 |
| 100 | 0.000218 | 0.0000777 |
| 1000 | 0.000354 | 0.0005381 |
| 10000 | 0.002702 | 0.0068388 |

The results show that runtime increases as the number of printed values increases, demonstrating that printing to the screen is a costly operation. Both C and Python exhibit this

trend, but the increase is more pronounced in Python. While Python appears competitive at very small input sizes, C scales more efficiently as the number of iterations grows. This highlights that input/output operations can dominate execution time and that compiled languages like C are generally more efficient for performance-critical tasks involving repeated operations. At small inputs (10, 100), Python is very fast and even appears faster than C . At larger inputs (1000, 10000), Python becomes significantly slower than C .

# Part 4

**What program does**

The programs used in this task create a file and write a specified number of lines to it, where the number of lines is determined by the input value. The file contains sequential text such as “line 1”, “line 2”, up to the chosen input number. A second set of programs then reads the contents of this file into memory and measures how long the operation takes. Both the writing and reading programs were implemented in Python and C. Each program uses timing functions to calculate the runtime of the core operation, allowing a comparison of file input/output performance between the two languages.

| Input size (lines) | C write time (s) | Python write time (s) |
|---|---|---|
| 10 | 0.000030 | 0.003099 |
| 100 | 0.000046 | 0.003746 |
| 1000 | 0.000211 | 0.004186 |
| 5000 | 0.000943 | 0.005330 |

| File size read | C read time (s) | Python read time (s) |
|---|---|---|
| 5000 lines | 0.000469 | 0.003897 |

The results show that file writing time increased as the number of lines increased in both C and Python. This is expected, since larger inputs mean more data must be written to disk. However, the C program was consistently much faster than the Python program for every input size tested.

For the write test, the C runtime increased from 0.000030 s for 10 lines to 0.000943 s for 5000 lines. The Python runtime also increased, from 0.003099 s to 0.005330 s, but it remained noticeably slower overall. Even for the smallest file size, Python had a much larger overhead.

For the read test, both programs successfully read 5000 lines into memory, but again C was much faster. The C program took 0.000469 s, while the Python program took 0.003897 s. This suggests that reading from a file also carries more overhead in Python than in C.

The file I/O experiments showed that execution time increased as the amount of data written to or read from a file increased. This trend was observed in both C and Python. In all cases, the C implementation was significantly faster than the Python implementation, showing the efficiency advantage of compiled code for file operations. The results also demonstrate that file input/output is more expensive than simple calculations, because it involves interaction

with the operating system and storage hardware. Overall, C gave the best performance, while Python was slower but still good for small files.